

tf.compat.v1.parse_example Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/parse_example

Parses Example protos into a dict of tensors.

Function Signatures

```
tf.compat.v1.parse_example( serialized, features, name=None,
example_names=None )
```

Code Examples

```
tf.compat.v1.parse_example(
    serialized, features, name=None, example_names=None
)
```

```
tf.compat.v1.parse_example(
    serialized, features, name=None, example_names=None
)
```

```
serialized = [  
  features  
  { feature { key: "ft" value { float_list { value: [1.0, 2.0] } } }  
},  
  features  
  { feature []},  
  features  
  { feature { key: "ft" value { float_list { value: [3.0] } } }  
]
```

```
serialized = [  
  features  
  { feature { key: "ft" value { float_list { value: [1.0, 2.0] } } }  
},  
  features  
  { feature []},  
  features  
  { feature { key: "ft" value { float_list { value: [3.0] } } }  
]
```

```
{"ft": SparseTensor(indices=[[0, 0], [0, 1], [2, 0]],  
  values=[1.0, 2.0, 3.0],  
  dense_shape=(3, 2)) }
```

```
{"ft": SparseTensor(indices=[[0, 0], [0, 1], [2, 0]],  
  values=[1.0, 2.0, 3.0],  
  dense_shape=(3, 2)) }
```

```
{"ft": [[1.0, 2.0], [3.0, -1.0]]}
```

```
{"ft": [[1.0, 2.0], [3.0, -1.0]]}
```

Documentation

Parses Example protos into a dict of tensors.

View aliases

Compat aliases for migration

See

Migration guide for
more details.

`tf.compat.v1.io.parse_example`

Parses a number of serialized Example
protos given in `serialized`. We refer to `serialized` as a batch with
`batch_size` many entries of individual Example protos.

`example_names` may contain descriptive names for the corresponding serialized
protos. These may be useful for debugging purposes, but they have no effect on
the output. If not `None`, `example_names` must be the same length as
`serialized`.

This op parses serialized examples into a dictionary mapping keys to Tensor
SparseTensor, and RaggedTensor objects. `features` is a Mapping from keys
to VarLenFeature, SparseFeature, RaggedFeature, and FixedLenFeature
objects. Each VarLenFeature and SparseFeature is mapped to a
SparseTensor; each FixedLenFeature is mapped to a Tensor; and each
RaggedFeature is mapped to a RaggedTensor.

Each VarLenFeature maps to a SparseTensor of the specified type
representing a ragged matrix. Its indices are `[batch, index]` where `batch`
identifies the example in `serialized`, and `index` is the value's index in

the list of values associated with that feature and example.

Each SparseFeature maps to a SparseTensor of the specified type representing a Tensor of dense_shape [batch_size] + SparseFeature.size.

Its values come from the feature in the examples with key value_key.

A values[i] comes from a position k in the feature of an example at batch entry batch. This positional information is recorded in indices[i] as

[batch, index_0, index_1, ...] where index_j is the k-th value of the feature in the example at with key SparseFeature.index_key[j].

In other words, we split the indices (except the first index indicating the batch entry) of a SparseTensor by dimension into different features of the Example. Due to its complexity a VarLenFeature should be preferred over a SparseFeature whenever possible.

Each FixedLenFeature df maps to a Tensor of the specified type (or tf.float32 if not specified) and shape (serialized.size(),) + df.shape.

FixedLenFeature entries with a default_value are optional. With no default value, we will fail if that Feature is missing from any example in serialized.

Each FixedLenSequenceFeature df maps to a Tensor of the specified type (or tf.float32 if not specified) and shape (serialized.size(), None) + df.shape.

All examples in serialized will be padded with default_value along the second dimension.

Each RaggedFeature maps to a RaggedTensor of the specified type. It is formed by stacking the RaggedTensor for each example, where the RaggedTensor for each individual example is constructed using the tensors specified by RaggedTensor.values_key and RaggedTensor.partition. See

the `tf.io.RaggedFeature` documentation for details and examples.

Examples:

For example, if one expects a `tf.float32 VarLenFeature` `ft` and three serialized Examples are provided:

then the output will look like:

If instead a `FixedLenSequenceFeature` with `default_value = -1.0` and `shape=[]` is used then the output will look like:

Given two Example input protos in serialized:

And arguments

Then the output is a dictionary:

For dense results in two serialized Examples:

We can use arguments:

And the expected output is:

An alternative to `VarLenFeature` to obtain a `SparseTensor` is `SparseFeature`. For example, given two Example input protos in serialized:

And arguments

Then the output is a dictionary:

See the `tf.io.RaggedFeature` documentation for examples showing how `RaggedFeature` can be used to obtain `RaggedTensors`.

Returns

`## Raises`